Open Computer Use - Comprehensive Repository Analysis

Repository: https://github.com/coasty-ai/open-computer-use.git

Analysis Date: March 2025

Executive Summary

Open Computer Use is a full-stack desktop application that enables AI-powered computer automation through a multi-agent system. The application combines a Next.js frontend, Python FastAPI backend, Electron desktop wrapper, and Docker containerization to provide a comprehensive computer use agent platform. The system allows users to interact with their computer through natural language commands, with AI agents executing tasks like clicking, typing, and navigating applications.

Table of Contents

1. Technology Stack Overview
2. Project Architecture
3. Root Directory Analysis
4. Frontend Analysis (app/ directory)
5. Backend Analysis (backend/ directory)
6. Component Library (components/ directory)
7. Electron Desktop App (electron/ directory)
8. Docker Configuration (docker/ directory)
9. Utility Libraries (lib/ directory)
10. Infrastructure (infrastructure/ directory)
11. Data Flow and Integration
12. Security Considerations

1. Technology Stack Overview

Next.js 14 | React 18 | TypeScript | Python 3.11 | FastAPI | Electron | Docker | Terraform | AWS | LangGraph | Claude AI | Tailwind CSS

Frontend Technologies

Technology	Version	Purpose
Next.js	14.x	React framework for server-side rendering and routing
React	18.x	UI component library
TypeScript	5.x	Type-safe JavaScript development
Tailwind CSS	3.x	Utility-first CSS framework
Radix UI	Latest	Accessible UI primitives
Framer Motion	Latest	Animation library

Backend Technologies

Technology	Version	Purpose
Python	3.11+	Core backend language
FastAPI	0.104+	Async web framework
LangGraph	Latest	Multi-agent orchestration
Anthropic Claude	Latest	AI model for computer use
Uvicorn	Latest	ASGI server
Pydantic	2.x	Data validation

2. Project Architecture

High-Level Architecture

```
┌─────────────────────────────────────────────────────────┐
│                    Electron Desktop App                 │
│  ┌──────────────────────────────────────────────────┐   │
│  │              Next.js Frontend (React)            │   │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐       │   │
│  │  │   Chat   │  │ Settings │  │ Display  │       │   │
│  │  │ Interface│  │  Panel   │  │ Viewer   │       │   │
│  │  └──────────┘  └──────────┘  └──────────┘       │   │
│  └──────────────────────────────────────────────────┘   │
│                          │                              │
│                          ▼                              │
│  ┌──────────────────────────────────────────────────┐   │
│  │              Python FastAPI Backend              │   │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐       │   │
│  │  │   Chat   │  │ Multi-   │  │ CUA      │       │   │
│  │  │ Endpoint │  │ Agent    │  │ Executor │       │   │
│  │  └──────────┘  └──────────┘  └──────────┘       │   │
│  └──────────────────────────────────────────────────┘   │
│                          │                              │
│                          ▼                              │
│  ┌──────────────────────────────────────────────────┐   │
│  │              External Services                   │   │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐       │   │
│  │  │ Claude   │  │   AWS    │  │ Docker   │       │   │
│  │  │   AI     │  │ Services │  │Containers│       │   │
│  │  └──────────┘  └──────────┘  └──────────┘       │   │
│  └──────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────┘
```

Key Architectural Patterns

- **Multi-Agent System:** Uses LangGraph for orchestrating multiple AI agents that can collaborate on complex tasks
- **Event-Driven Communication:** WebSocket-based real-time communication between frontend and backend
- **Containerized Deployment:** Docker-based deployment for consistent environments
- **Desktop-First Design:** Electron wrapper provides native desktop integration

3. Root Directory Analysis

package.json

The root package.json serves as the primary configuration for the Next.js frontend application. It defines all client-side dependencies, build scripts, and development configurations.

Key Scripts:

- `dev` - Starts Next.js development server
- `build` - Builds the production application
- `start` - Runs the production server
- `lint` - Runs ESLint for code quality

Key Dependencies:

```
{
  "next": "^14.x",
  "react": "^18.x",
  "react-dom": "^18.x",
  "tailwindcss": "^3.x",
  "@radix-ui/react-*": "Various UI components",
  "framer-motion": "Animation library",
  "ai": "Vercel AI SDK",
  "marked": "Markdown parsing"
}
```

next.config.js

Configures Next.js with custom settings including output optimization for Electron integration, image handling, and experimental features for server actions.

tailwind.config.js

Defines Tailwind CSS configuration with custom color schemes, animation keyframes, and content paths for style generation.

tsconfig.json

TypeScript configuration with strict type checking enabled, path aliases for clean imports, and Next.js-specific compiler options.

4. Frontend Analysis (app/ directory)

The `app/` directory follows Next.js 14's App Router pattern, providing file-based routing with server components and client-side interactivity.

Directory Structure

```
app/
├── layout.tsx          # Root layout wrapper
├── page.tsx            # Home page
├── globals.css         # Global styles
├── api/                # API route handlers
│   └── chat/           # Chat API endpoint
│       └── route.ts    # Chat route handler
├── auth/               # Authentication pages
│   └── callback/       # OAuth callback handler
└── (main)/             # Route group for main app
    └── chat/           # Chat interface pages
```

app/layout.tsx

The root layout component that wraps all pages. It provides:

- HTML structure with proper metadata
- Global CSS imports including Tailwind
- Font configuration (Inter for UI, JetBrains Mono for code)
- Theme provider wrapper for dark/light mode

app/page.tsx

The landing page that serves as the entry point. It typically displays:

- Welcome screen with application introduction
- Quick start guide for new users
- Navigation to the main chat interface

app/api/chat/route.ts

The API route handler for chat functionality. This endpoint:

- Accepts POST requests with user messages
- Communicates with the Python backend
- Returns streaming responses for real-time output
- Handles error states and connection failures

app/globals.css

Global stylesheet with:

- CSS custom properties for theming
- Tailwind base, components, and utilities layers
- Custom animations and transitions
- Scroll behavior and scrollbar styling

5. Backend Analysis (backend/ directory)

The `backend/` directory contains the Python FastAPI application that powers the AI agent system.

Directory Structure

```
backend/
├── main.py                 # FastAPI application entry point
├── requirements.txt        # Python dependencies
├── app/
│   ├── __init__.py
│   ├── main.py            # App configuration
│   ├── api/
│   │   ├── __init__.py
│   │   ├── deps.py        # Dependency injection
│   │   └── routes/
│   │       ├── __init__.py
│   │       └── chat.py    # Chat endpoint
│   ├── services/
│   │   ├── __init__.py
│   │   └── multi_agent_executor.py  # Agent orchestration
│   └── models/
│       └── __init__.py    # Pydantic models
└── cua_executor/          # Computer Use Agent executor
    ├── __init__.py
    ├── executor.py        # Main executor logic
    └── tools.py           # Computer interaction tools
```

backend/main.py

The main entry point for the FastAPI server. It:

- Initializes the FastAPI application with CORS middleware
- Sets up logging configuration
- Includes API routers from various modules
- Configures startup and shutdown events
- Defines the server host and port settings

Key Configuration:

```
app = FastAPI(
    title="Open Computer Use API",
    description="AI-powered computer automation API",
    version="1.0.0"
)

# CORS configuration for frontend integration
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

backend/app/api/routes/chat.py

The core chat endpoint that handles user interactions:

- Accepts chat messages with conversation history
- Processes requests through the multi-agent executor
- Streams responses back to the client
- Manages session state and context

Endpoint Structure:

```
@router.post("/chat")
async def chat(request: ChatRequest):
    """
    Process chat message and return AI response.

    Args:
        request: ChatRequest with message and history

    Returns:
        StreamingResponse with AI output
    """
    # Validate input
    # Initialize agent executor
    # Stream response chunks
    # Handle errors and cleanup
```

backend/app/services/multi_agent_executor.py

The sophisticated multi-agent orchestration system that:

- Uses LangGraph for agent workflow management
- Coordinates multiple AI agents for complex tasks
- Implements agent-to-agent communication
- Manages task delegation and result aggregation
- Provides error handling and retry logic

Agent Types:

Agent	Role
Coordinator	Routes tasks to appropriate specialist agents
Computer Use	Executes mouse/keyboard actions on screen
Analysis	Interprets screen content and provides insights

backend/requirements.txt

Python dependencies for the backend:

```
fastapi>=0.104.0
uvicorn[standard]>=0.24.0
langgraph>=0.0.20
anthropic>=0.18.0
pydantic>=2.0.0
python-multipart>=0.0.6
websockets>=12.0
aiohttp>=3.9.0
pillow>=10.0.0
pyautogui>=0.9.54
python-dotenv>=1.0.0
```

backend/cua_executor/

The Computer Use Agent (CUA) executor module contains the core logic for screen interaction:

- **executor.py:** Main execution engine that processes AI commands and translates them into screen actions
- **tools.py:** Collection of tools for mouse movement, clicking, typing, scrolling, and screenshot capture

Available Tools:

Tool	Function
mouse_move	Move cursor to coordinates
mouse_click	Click at current or specified position
keyboard_type	Type text string
keyboard_hotkey	Press key combinations
scroll	Scroll in specified direction
screenshot	Capture current screen state

6. Component Library (components/ directory)

The `components/` directory houses reusable React components built with Radix UI primitives and styled with Tailwind CSS.

Directory Structure

```
components/
├── ui/                     # Base UI primitives
│   ├── button.tsx        # Button component
│   ├── input.tsx         # Input fields
│   ├── dialog.tsx        # Modal dialogs
│   ├── dropdown-menu.tsx # Dropdown menus
│   ├── scroll-area.tsx   # Scrollable containers
│   ├── separator.tsx     # Visual separators
│   └── tooltip.tsx       # Tooltip overlays
├── chat/                  # Chat-specific components
│   ├── chat-container.tsx    # Main chat wrapper
│   ├── chat-input.tsx        # Message input area
│   ├── chat-message.tsx      # Individual message display
│   ├── chat-messages.tsx     # Message list container
│   └── chat-suggestions.tsx  # Quick action suggestions
├── sidebar/                # Navigation sidebar
│   ├── sidebar.tsx       # Main sidebar component
│   └── sidebar-items.tsx # Navigation items
└── theme/                  # Theme controls
    └── theme-toggle.tsx  # Dark/light mode toggle
```

components/ui/

Base UI components built on Radix UI primitives:

- **button.tsx:** Customizable button with variants (default, destructive, outline, ghost, link)
- **input.tsx:** Styled input field with focus states
- **dialog.tsx:** Modal dialog system with overlay and close functionality
- **dropdown-menu.tsx:** Context menu and dropdown navigation
- **scroll-area.tsx:** Custom scrollbar styling for content areas

components/chat/

Chat interface components for user-AI interaction:

- **chat-container.tsx:** Root wrapper managing chat state and layout
- **chat-input.tsx:** Text input with send button, keyboard shortcuts, and file attachment support
- **chat-message.tsx:** Individual message bubble with markdown rendering and code highlighting
- **chat-messages.tsx:** Scrollable list with auto-scroll to latest messages
- **chat-suggestions.tsx:** Quick action buttons for common tasks

components/sidebar/

Navigation sidebar components:

- Conversation history list
- New chat button
- Settings access
- Collapsible design for space efficiency

7. Electron Desktop App (electron/ directory)

The `electron/` directory wraps the Next.js application as a native desktop application with system integration.

Directory Structure

```
electron/
├── package.json          # Electron-specific dependencies
├── main.js               # Main Electron process
├── preload.js            # Bridge between main and renderer
└── electron-builder.yml  # Build configuration
```

electron/package.json

Electron application configuration with dependencies:

```
{
  "name": "open-computer-use",
  "version": "1.0.0",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "build": "electron-builder",
    "build:mac": "electron-builder --mac",
    "build:win": "electron-builder --win",
    "build:linux": "electron-builder --linux"
  },
  "dependencies": {
    "electron": "^28.x",
    "electron-builder": "^24.x"
  }
}
```

electron/main.js

The main Electron process handles:

- BrowserWindow creation and management
- Loading the Next.js application (development server or built files)
- IPC (Inter-Process Communication) setup
- Application lifecycle events (ready, quit, activate)
- Menu and tray icon management
- Native dialog handling

Window Configuration:

```
const mainWindow = new BrowserWindow({
  width: 1200,
  height: 800,
  minWidth: 800,
  minHeight: 600,
  webPreferences: {
    nodeIntegration: false,
    contextIsolation: true,
    preload: path.join(__dirname, 'preload.js')
  },
  titleBarStyle: 'hiddenInset',
  trafficLightPosition: { x: 15, y: 15 }
});
```

electron/preload.js

The preload script provides a secure bridge between the renderer process and main process:

- Exposes limited API to renderer via contextBridge
- Handles secure IPC communication
- Provides access to native features (file system, dialogs)

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
  selectDirectory: () => ipcRenderer.invoke('select-directory'),
  getAppPath: () => ipcRenderer.invoke('get-app-path'),
  openExternal: (url) => ipcRenderer.invoke('open-external', url),
  // ... more exposed methods
});
```

electron/electron-builder.yml

Build configuration for packaging the application:

```yaml
appId: com.coasty.open-computer-use
productName: Open Computer Use
directories:
  buildResources: build
  output: dist
mac:
  category: public.app-category.productivity
  target: [dmg, zip]
win:
  target: [nsis, portable]
linux:
  target: [AppImage, deb]
```

8. Docker Configuration (docker/ directory)

The `docker/` directory contains containerization configurations for development and deployment.

Directory Structure

```
docker/
├── ai-desktop/
│   ├── Dockerfile        # Main application container
│   └── docker-compose.yml # Multi-service orchestration
└── backend/
    └── Dockerfile        # Backend-only container
```

docker/ai-desktop/Dockerfile

Multi-stage Dockerfile for the complete application:

```
# Stage 1: Frontend build
FROM node:20-alpine AS frontend-builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Python backend
FROM python:3.11-slim AS backend
WORKDIR /app
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ .

# Stage 3: Final image
FROM node:20-slim
# Install dependencies, copy artifacts, configure runtime
EXPOSE 3000
CMD ["npm", "start"]
```

docker/ai-desktop/docker-compose.yml

Multi-service orchestration configuration:

```yaml
version: '3.8'
services:
  frontend:
    build:
      context: ../
      dockerfile: docker/ai-desktop/Dockerfile
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
    depends_on:
      - backend

  backend:
    build:
      context: ../backend
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
    volumes:
      - ./data:/app/data
```

9. Utility Libraries (lib/ directory)

The `lib/` directory contains shared utilities, API clients, and helper functions.

Directory Structure

```
lib/
├── api.ts                # API client functions
├── utils.ts              # General utility functions
├── constants.ts          # Application constants
├── types.ts              # TypeScript type definitions
└── hooks/                # Custom React hooks
    ├── use-chat.ts       # Chat state management
    ├── use-settings.ts   # Settings persistence
    └── use-theme.ts      # Theme management
```

lib/api.ts

API client for backend communication:

- Chat message submission
- Streaming response handling
- Error handling and retry logic
- Authentication header management

```
export async function sendChatMessage(
  message: string,
  history: Message[]
): Promise {
  const response = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message, history })
  });

  if (!response.ok) {
    throw new Error('Chat request failed');
  }

  return response.body;
}
```

lib/utils.ts

General utility functions including:

- `cn()` - Class name merger using clsx and tailwind-merge
- Date formatting helpers
- String manipulation utilities
- Validation functions

```
import { clsx, type ClassValue } from 'clsx';
import { twMerge } from 'tailwind-merge';

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

lib/hooks/use-chat.ts

Custom hook for chat state management:

- Message history state
- Loading states
- Error handling
- Message submission logic

10. Infrastructure (infrastructure/ directory)

The `infrastructure/` directory contains Infrastructure as Code (IaC) configurations for cloud deployment.

Directory Structure

```
infrastructure/
├── main.tf              # Main Terraform configuration
├── variables.tf         # Input variables
├── outputs.tf           # Output definitions
├── providers.tf         # Cloud provider configuration
└── modules/             # Reusable Terraform modules
    ├── compute/         # EC2/ECS configurations
    ├── networking/      # VPC, subnets, security groups
    └── storage/         # S3, RDS configurations
```

infrastructure/main.tf

Main Terraform configuration defining AWS resources:

- VPC and networking infrastructure
- ECS cluster for container orchestration
- Application Load Balancer
- CloudWatch for logging and monitoring
- IAM roles and policies

infrastructure/variables.tf

Configurable input variables:

```
variable "aws_region" {
  description = "AWS region for deployment"
  default     = "us-west-2"
}

variable "environment" {
  description = "Deployment environment"
  default     = "production"
}

variable "instance_count" {
  description = "Number of application instances"
  default     = 2
}
```

infrastructure/providers.tf

Cloud provider configuration:

```
provider "aws" {
  region = var.aws_region
}

terraform {
  backend "s3" {
    bucket = "open-computer-use-terraform"
    key    = "infrastructure.tfstate"
    region = "us-west-2"
  }
}
```

11. Data Flow and Integration

Request Flow Diagram

```
User Input → Chat Component → API Route → Backend
                                          ↓
                                    Multi-Agent Executor
                                          ↓
                                    Claude AI API
                                          ↓
                                    CUA Executor
                                          ↓
                                    Screen Actions
                                          ↓
                                    Response Stream → UI Update
```

Key Integration Points

1. **Frontend-Backend Communication:** REST API with streaming responses for real-time updates
2. **AI Model Integration:** Anthropic Claude API for computer use capabilities
3. **Screen Control:** PyAutoGUI and PIL for mouse, keyboard, and screenshot operations
4. **Desktop Integration:** Electron IPC for native OS features

12. Security Considerations

Important Security Notes

- API keys should be stored securely in environment variables
- The application requires screen access permissions on macOS and Windows
- Electron's contextIsolation is enabled for renderer security
- Input validation is handled by Pydantic models in the backend

Security Features

Feature	Implementation
API Key Protection	Environment variables, never committed to code
CORS	Configured for specific origins in production
Input Validation	Pydantic models with strict typing
Electron Security	contextIsolation, nodeIntegration disabled
Rate Limiting	Implemented at API gateway level

Conclusion

Open Computer Use is a sophisticated, full-stack application that demonstrates modern software architecture patterns. The combination of Next.js frontend, Python FastAPI backend, Electron desktop wrapper, and Docker containerization provides a robust foundation for AI-powered computer automation.

Key Strengths

- Modular architecture with clear separation of concerns
- Modern tech stack with TypeScript, React 18, and Python 3.11
- Multi-agent orchestration using LangGraph
- Cross-platform desktop support via Electron
- Cloud-ready with Docker and Terraform configurations

Potential Improvements

- Add comprehensive test coverage (unit, integration, e2e)
- Implement user authentication and session management
- Add more detailed API documentation (OpenAPI/Swagger)
- Implement audit logging for security compliance

Document generated for NinjaTech AI - Repository Analysis